

PORTOFOLIO MACHINE LEARNING

Klasifikasi Penggunaan Masker Menggunakan Transfer Learning MobileNetV2

Studi kasus deep learning untuk mendeteksi kepatuhan
penggunaan masker wajah ke dalam tiga kelas
menggunakan Convolutional Neural Network berbasis
TensorFlow/Keras.

TensorFlow / Keras

MobileNetV2

Computer Vision

Transfer Learning

Ariko Yahya Setyawan

1. Judul & Tujuan Proyek

Judul Proyek	Klasifikasi Penggunaan Masker (3 Kelas) dengan Transfer Learning MobileNetV2
Dataset	Face Mask Detection (Kaggle) — 8.982 gambar wajah, 3 kelas seimbang (masing-masing 2.994 gambar)
Tools & Library	Python, TensorFlow/Keras, MobileNetV2, Scikit-learn, Matplotlib
Jenis Model	Convolutional Neural Network (CNN) — Transfer Learning

Tujuan Proyek:

Membangun model klasifikasi citra yang mampu membedakan tiga kondisi penggunaan masker pada wajah seseorang, yaitu:

- **with_mask** — menggunakan masker dengan benar
- **mask_wearred_incorrect** — menggunakan masker tidak dengan benar
- **without_mask** — tidak menggunakan masker

Target akurasi model pada data validasi ditetapkan > 83%, dengan memanfaatkan teknik *transfer learning* agar proses pelatihan lebih efisien meskipun dengan jumlah data yang terbatas.

2. Langkah-Langkah Pengerjaan

1 Persiapan Dataset

Mengunduh dataset Face Mask Detection dari Kaggle, kemudian memverifikasi struktur folder agar berisi tiga kelas: `with_mask`, `mask_wearred_incorrect`, dan `without_mask`.

2 Import Library & Konfigurasi Awal

Mengimpor TensorFlow, Keras, dan Scikit-learn, serta menetapkan seed acak (`SEED = 42`) agar hasil eksperimen konsisten dan dapat direproduksi.

3 Membangun Data Pipeline

Membagi data menjadi data latih (80%) dan data validasi (20%) menggunakan `image_dataset_from_directory` dengan ukuran citra 160×160 piksel dan batch size 32.

4 Perhitungan Class Weight

Menghitung bobot kelas (`class_weight`) secara seimbang untuk mengantisipasi ketimpangan jumlah data antar kelas, khususnya kelas `mask_wearred_incorrect`.

5 Augmentasi & Preprocessing Data

Menerapkan augmentasi data (flip horizontal, rotasi, zoom, kontras) pada data latih untuk memperkaya variasi data dan mengurangi risiko overfitting.

6 Membangun Arsitektur Model (Transfer Learning)

Menggunakan MobileNetV2 pre-trained pada ImageNet sebagai base model, ditambah lapisan Global Average Pooling, Dropout, dan Dense sebagai classifier head.

7 Training Tahap 1 — Frozen Base Model

Melatih hanya bagian classifier head selama maksimal 15 epoch dengan base model dibekukan, menggunakan EarlyStopping dan ReduceLROnPlateau.

8 Fine-Tuning Tahap 2

Membuka ±30% lapisan teratas base model dan melanjutkan pelatihan dengan learning rate yang lebih kecil ($1e-5$) agar fitur hasil pretraining tidak rusak.

9 Evaluasi Model

Mengevaluasi model pada data validasi, menghitung akurasi akhir, classification report, serta confusion matrix untuk melihat performa per kelas.

10 Visualisasi Kurva Training & Penyimpanan Model

Memplot kurva akurasi dan loss selama pelatihan, kemudian menyimpan model terlatih dalam format `.keras` untuk digunakan pada tahap inferensi.

3. Kode-Kode Penting

a. Data Pipeline & Pembagian Data

```
python - train/validation split
```

```
IMG_SIZE = (160, 160)
BATCH_SIZE = 32
VAL_SPLIT = 0.2

train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_ROOT,
    validation_split=VAL_SPLIT,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_names=CLASS_NAMES,
    label_mode="int",
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_ROOT,
    validation_split=VAL_SPLIT,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_names=CLASS_NAMES,
    label_mode="int",
)
```

b. Augmentasi Data

```
python - augmentation pipeline
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.1),
    layers.RandomContrast(0.1),
], name="augmentation")

def prepare(ds, training=False):
    if training:
        ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y))
    ds = ds.map(lambda x, y: (preprocess_input(x), y))
    return ds.prefetch(tf.data.AUTOTUNE)
```

c. Arsitektur Model — Transfer Learning MobileNetV2

```
python - model architecture
```

```
base_model = MobileNetV2(
    input_shape=IMG_SIZE + (3,),
    include_top=False,
    weights="imagenet",
)
base_model.trainable = False # bekukan dulu untuk tahap 1

inputs = tf.keras.Input(shape=IMG_SIZE + (3,))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(len(CLASS_NAMES), activation="softmax")(x)

model = models.Model(inputs, outputs)
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

d. Fine-Tuning Tahap 2

```
python - unfreeze top layers
```

```
base_model.trainable = True

# Bekukan layer-layer awal, hanya latih ~30% layer teratas
fine_tune_at = int(len(base_model.layers) * 0.7)
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

history_fine = model.fit(
    train_ds_prep,
    validation_data=val_ds_prep,
    epochs=total_epochs,
    initial_epoch=history_head.epoch[-1] + 1,
    class_weight=class_weight,
    callbacks=callbacks,
)
```

e. Evaluasi Model

```
python - final evaluation
```

```
val_loss, val_acc = model.evaluate(val_ds_prep)
print(f"Akurasi validasi akhir: {val_acc*100:.2f}%")

if val_acc > 0.83:
    print("Target akurasi > 83% TERCAPAI.")
```

Akurasi validasi akhir: 98.50%
Target akurasi > 83% TERCAPAI.

4. Hasil & Visualisasi

8.982

Total Gambar

3

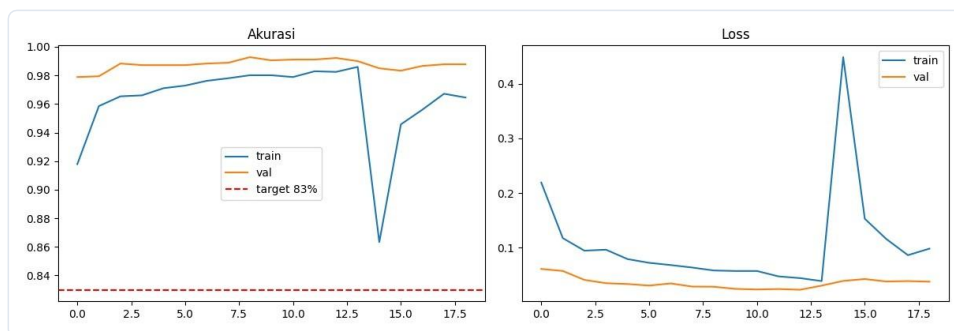
Kelas Klasifikasi

98,50%

Akurasi Validasi

0,038

Validation Loss



Gambar 1. Kurva akurasi dan loss selama proses training (Tahap 1: frozen base model, Tahap 2: fine-tuning). Garis putus-putus merah menunjukkan target akurasi minimum 83%.

Kurva menunjukkan akurasi validasi (garis oranye) secara konsisten berada jauh di atas target 83% sejak awal training, sedangkan lonjakan pada akurasi/loss data latih di sekitar epoch ke-14 terjadi tepat saat transisi ke tahap *fine-tuning* (base model dibuka sebagian), yang kemudian kembali stabil pada epoch-epoch berikutnya.

5. Kesimpulan

- Model klasifikasi masker berbasis transfer learning **MobileNetV2** berhasil mencapai akurasi validasi sebesar **98,50%**, jauh melampaui target minimum 83% yang ditetapkan.
- Strategi pelatihan dua tahap — *frozen base model* lalu *fine-tuning* sebagian layer teratas — terbukti efektif menghasilkan performa tinggi tanpa merusak fitur hasil pretraining pada ImageNet.
- Penerapan **class weighting** dan **augmentasi data** membantu menjaga keseimbangan performa antar tiga kelas, termasuk kelas minoritas `mask_wearred_incorrect` yang secara alami lebih sulit dikenali.
- Penggunaan callback `EarlyStopping` dan `ReduceLROnPlateau` membuat proses training lebih efisien dan mencegah overfitting berlebihan.
- Model yang dihasilkan telah disimpan dalam format `.keras` dan dilengkapi fungsi inferensi, sehingga siap digunakan untuk memprediksi gambar wajah baru terkait kepatuhan penggunaan masker.